

CSE312: Software Architecture — Principles and Practices
Course Project — UML System Design Documentation

LexFlow
AI-Powered Legal Case Assistant & Case Management System

Use Case • Class • Sequence • Activity • State Diagrams

Submitted By

Name	Roll No.
Yashwanth S	2024BCS0264
Badagala Bharath Kumar	2024BCS0160
G Harshavardhan S R Gopal	2024BCD0044
Mood Anand Naik	2024BCS0076

1. Project Overview

Project Name: LexFlow

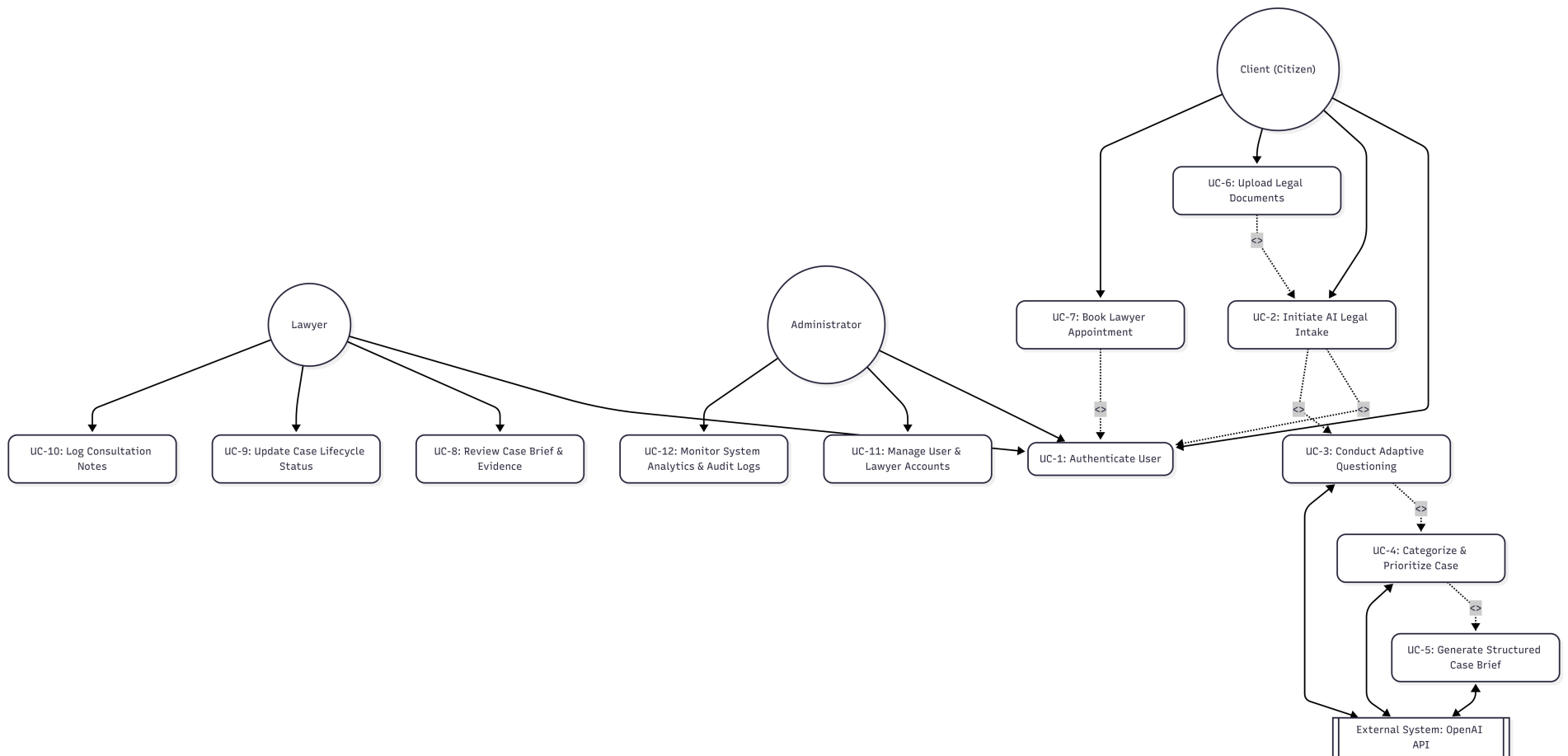
Tagline: AI-Powered Legal Case Assistant & Case Management System

LexFlow is an AI-powered legal intake and case management system designed to streamline communication and data collection between citizens and legal professionals. The platform uses conversational AI to guide clients through describing their legal issues, dynamically gathering essential details, categorizing disputes, and generating structured briefs without rendering legal advice. Lawyers receive pre-organized summaries, document repositories, and scheduling tools, while administrators oversee platform compliance and audit logs. LexFlow significantly reduces initial consultation preparation time and improves overall legal service accessibility.

2. Use Case Diagram — Actors

Actor	Role Description
Client (Citizen)	An end-user seeking legal assistance who interacts with the AI intake agent, uploads evidence documents, and books consultations.
Lawyer	A verified legal practitioner who reviews AI-generated briefs, manages appointments, updates case lifecycle milestones, and records consultation notes.
Administrator	A system operator responsible for user account management, lawyer verification, monitoring system analytics, and inspecting audit logs.
External System: OpenAI API	An external LLM service integrated via REST API that processes conversational dialogue, extracts legal entities, and formats structured case briefs.

Figure 1: LexFlow — Use Case Diagram



3. Use Case Diagram — Use Cases List

Use Cases & Participating Actors

ID	Use Case	Participating Actor(s)
UC-1	Authenticate User	Client, Lawyer, Admin
UC-2	Initiate AI Legal Intake	Client, External System: OpenAI API
UC-3	Conduct Adaptive Questioning	Client, External System: OpenAI API
UC-4	Categorize & Prioritize Case	External System: OpenAI API
UC-5	Generate Structured Case Brief	External System: OpenAI API
UC-6	Upload Legal Documents	Client
UC-7	Book Lawyer Appointment	Client, Lawyer
UC-8	Review Case Brief & Evidence	Lawyer
UC-9	Update Case Lifecycle Status	Lawyer
UC-10	Log Consultation Notes	Lawyer
UC-11	Manage User & Lawyer Accounts	Administrator
UC-12	Monitor System Analytics & Audit Logs	Administrator

Explicit <<include>> and <<extend>> Relationships

Type	Relationship	Reason
<<include>>	UC-2 → UC-1	User authentication is mandatory prior to initializing an active legal intake session.
<<include>>	UC-2 → UC-3	Conducting adaptive questioning is an essential sub-process required to gather complete intake information.
<<include>>	UC-3 → UC-4	Legal case categorization and priority assignment are automatically executed upon completing dialogue fact-gathering.

<<include>>	UC-4 → UC-5	Generating the structured case brief is automatically performed immediately following categorization and prioritization.
<<include>>	UC-7 → UC-1	Valid user authentication must be verified before confirming an appointment booking.
<<extend>>	UC-6 → UC-2	Uploading evidence documents is an optional task performed by clients to complement their conversational AI intake session.

4. Use Case Descriptions

UC-1: Authenticate User

Purpose:	Validates user credentials (email/password) and issues a role-based JSON Web Token (JWT) for secure platform access.
Actor(s):	Client, Lawyer, Administrator
Expected Outcome:	User is authenticated, granted access to role-appropriate dashboards, and assigned an encrypted session token.

UC-2: Initiate AI Legal Intake

Purpose:	Provides a conversational chat interface for clients to describe their legal issue in natural language.
Actor(s):	Client, External System (OpenAI API)
Expected Outcome:	Client's initial statement is recorded, and an active intake session is initialized without generating direct legal advice.

UC-3: Conduct Adaptive Questioning

Purpose:	Dynamically analyzes client input to ask relevant clarification questions and gather missing factual details such as dates, parties, and grievances.
Actor(s):	Client, External System (OpenAI API)
Expected Outcome:	Essential case facts are interactively collected and recorded within the active conversation memory.

UC-4: Categorize & Prioritize Case

Purpose:	Evaluates extracted case facts to assign a legal category (e.g., Property, Family, Criminal) and an urgency rating (Low, Medium, High, Urgent).
Actor(s):	External System (OpenAI API)
Expected Outcome:	The legal issue is tagged with a standard domain classification and urgency level for downstream routing.

UC-5: Generate Structured Case Brief

Purpose:	Compiles raw intake dialogue into a standardized executive brief containing incident timelines, involved parties, core claims, and required evidence.
Actor(s):	External System (OpenAI API)
Expected Outcome:	A structured JSON summary is created and stored in the database for lawyer review.

UC-6: Upload Legal Documents

Purpose:	Allows clients to upload evidentiary files (PDFs, images, notices, agreements) associated with their intake session.
Actor(s):	Client
Expected Outcome:	Documents are validated for type and size, encrypted, and attached to the corresponding case record.

UC-7: Book Lawyer Appointment

Purpose:	Enables clients to select a lawyer matching their case category and reserve an available consultation slot.
Actor(s):	Client, Lawyer
Expected Outcome:	An appointment record is created in Requested or Confirmed status, and calendar notifications are triggered.

UC-8: Review Case Brief & Evidence

Purpose:	Gives assigned lawyers access to the AI-generated structured brief, extracted entity tables, and uploaded client documents prior to consultation.
Actor(s):	Lawyer
Expected Outcome:	Lawyer reviews pre-organized case facts, reducing manual intake time during consultations.

UC-9: Update Case Lifecycle Status

Purpose:	Tracks and transitions the legal dispute across predefined operational milestones.
Actor(s):	Lawyer

Expected Outcome:	The case status updates sequentially (e.g., from Consultation Scheduled to Case Filed or Closed) and updates client tracking views.
--------------------------	---

UC-10: Log Consultation Notes

Purpose:	Allows lawyers to enter confidential strategy notes, court filing details, and follow-up actions following client interactions.
Actor(s):	Lawyer
Expected Outcome:	Consultation notes are appended to the permanent case audit trail.

UC-11: Manage User & Lawyer Accounts

Purpose:	Provides administrative control over account approvals, lawyer credentials verification, role assignments, and account suspensions.
Actor(s):	Administrator
Expected Outcome:	Platform user access remains verified, secure, and compliant with organizational standards.

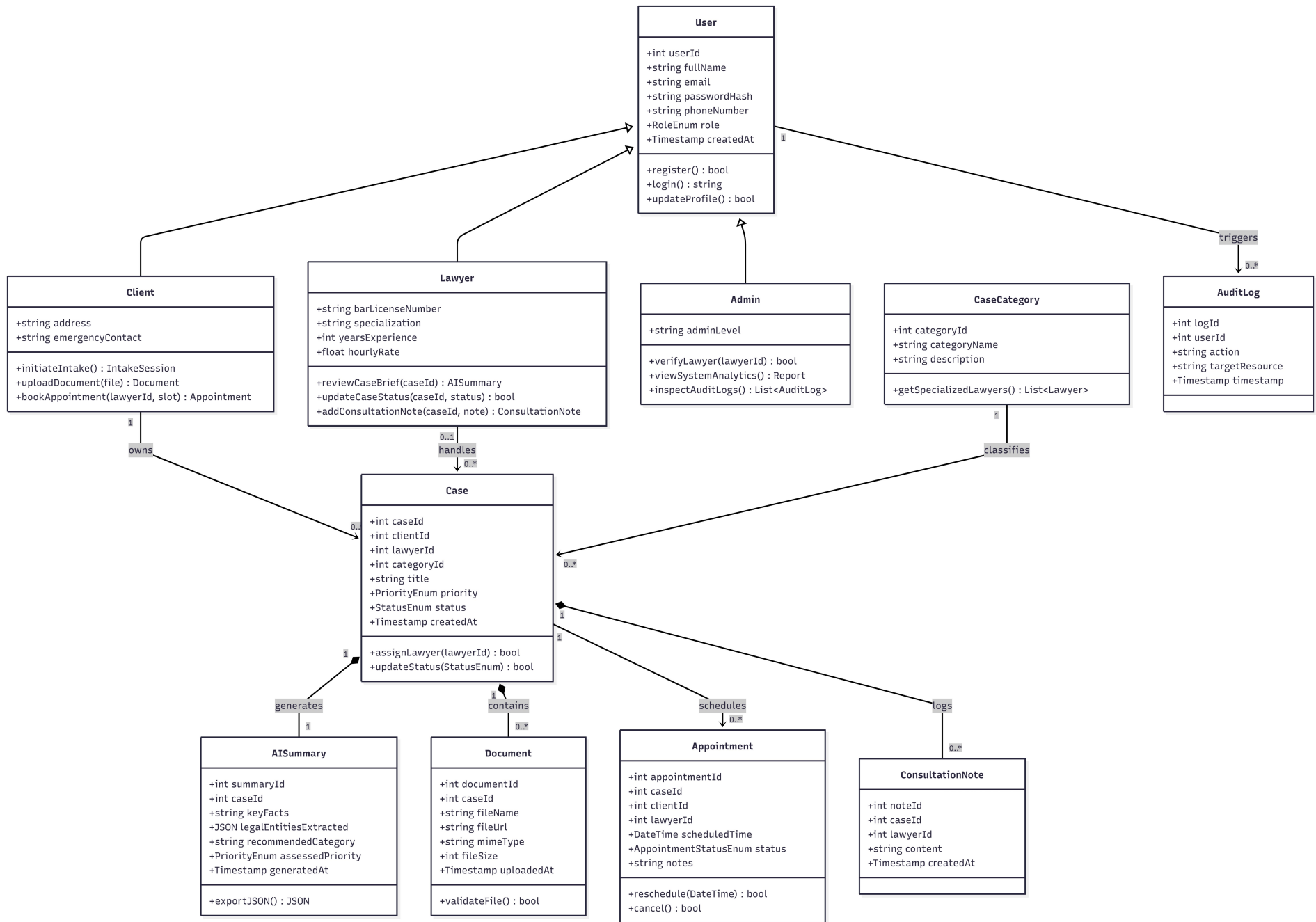
UC-12: Monitor System Analytics & Audit Logs

Purpose:	Displays platform-wide usage metrics, triage response times, category distributions, and immutable security audit logs.
Actor(s):	Administrator
Expected Outcome:	System performance and data access patterns are audited for operational safety and legal compliance.

5. Class Diagram

The class diagram below models the core domain entities of LexFlow: the **User** hierarchy (Client, Lawyer, Admin), the central **Case** aggregate, and its associated **AISummary**, **Document**, **Appointment**, **ConsultationNote**, **CaseCategory**, and **AuditLog** entities, along with their attributes, methods, and relationships.

Figure 2: LexFlow — Class Diagram



5.1 Class Diagram — Class Details

User (Base Class)	
Attributes	userId: int, fullName: string, email: string, passwordHash: string, phoneNumber: string, role: RoleEnum, createdAt: Timestamp
Methods	register(): bool, login(): string, updateProfile(): bool
Relationships	Base class for Client, Lawyer, Admin (Generalization). 1 User triggers 0..* AuditLog (Association).

Client (extends User)	
Attributes	address: string, emergencyContact: string
Methods	initiateIntake(): IntakeSession, uploadDocument(file): Document, bookAppointment(lawyerId, slot): Appointment
Relationships	Inherits User. 1 Client owns 0..* Case. 1 Client schedules 0..* Appointment.

Lawyer (extends User)	
Attributes	barLicenseNumber: string, specialization: string, yearsExperience: int, hourlyRate: float
Methods	reviewCaseBrief(caseId): AISummary, updateCaseStatus(caseId, status): bool, addConsultationNote(caseId, note): ConsultationNote
Relationships	Inherits User. 0..1 Lawyer handles 0..* Case. 1 Lawyer receives 0..* Appointment.

Admin (extends User)	
Attributes	adminLevel: string
Methods	verifyLawyer(lawyerId): bool, viewSystemAnalytics(): Report, inspectAuditLogs(): List<AuditLog>
Relationships	Inherits User.

Case	
Attributes	caseId: int, clientId: int, lawyerId: int, categoryId: int, title: string, priority: PriorityEnum, status: StatusEnum, createdAt: Timestamp
Methods	assignLawyer(lawyerId): bool, updateStatus(newStatus): bool
Relationships	Owned by 1 Client (1..0..*). Handled by 0..1 Lawyer. Classified by 1 CaseCategory. Composes AISummary (1:1), Document (1:0..*), ConsultationNote (1:0..*). Associated with Appointment (1:0..*).

CaseCategory	
--------------	--

Attributes	categoryId: int, categoryName: string, description: string
Methods	getSpecializedLawyers(): List<Lawyer>
Relationships	Classifies 0..* Case (1 to 0..*).

AI Summary

Attributes	summaryId: int, caseId: int, keyFacts: string, legalEntitiesExtracted: JSON, recommendedCategory: string, assessedPriority: PriorityEnum, generatedAt: Timestamp
Methods	exportJSON(): JSON
Relationships	Part of 1 Case (Composition, 1:1).

Document

Attributes	documentId: int, caseId: int, fileName: string, fileUrl: string, mimeType: string, fileSize: int, uploadedAt: Timestamp
Methods	validateFile(): bool
Relationships	Part of 1 Case (Composition, 1:0..*).

Appointment

Attributes	appointmentId: int, caseId: int, clientId: int, lawyerId: int, scheduledTime: DateTime, status: AppointmentStatusEnum, notes: string
Methods	reschedule(newTime): bool, cancel(): bool
Relationships	Scheduled for 1 Case. Booked by 1 Client. Assigned to 1 Lawyer (each 1:0..*).

ConsultationNote

Attributes	noteId: int, caseId: int, lawyerId: int, content: string, createdAt: Timestamp
Methods	—
Relationships	Part of 1 Case (Composition, 1:0..*).

AuditLog

Attributes	logId: int, userId: int, action: string, targetResource: string, timestamp: Timestamp
Methods	—
Relationships	Triggered by 1 User (1:0..*).

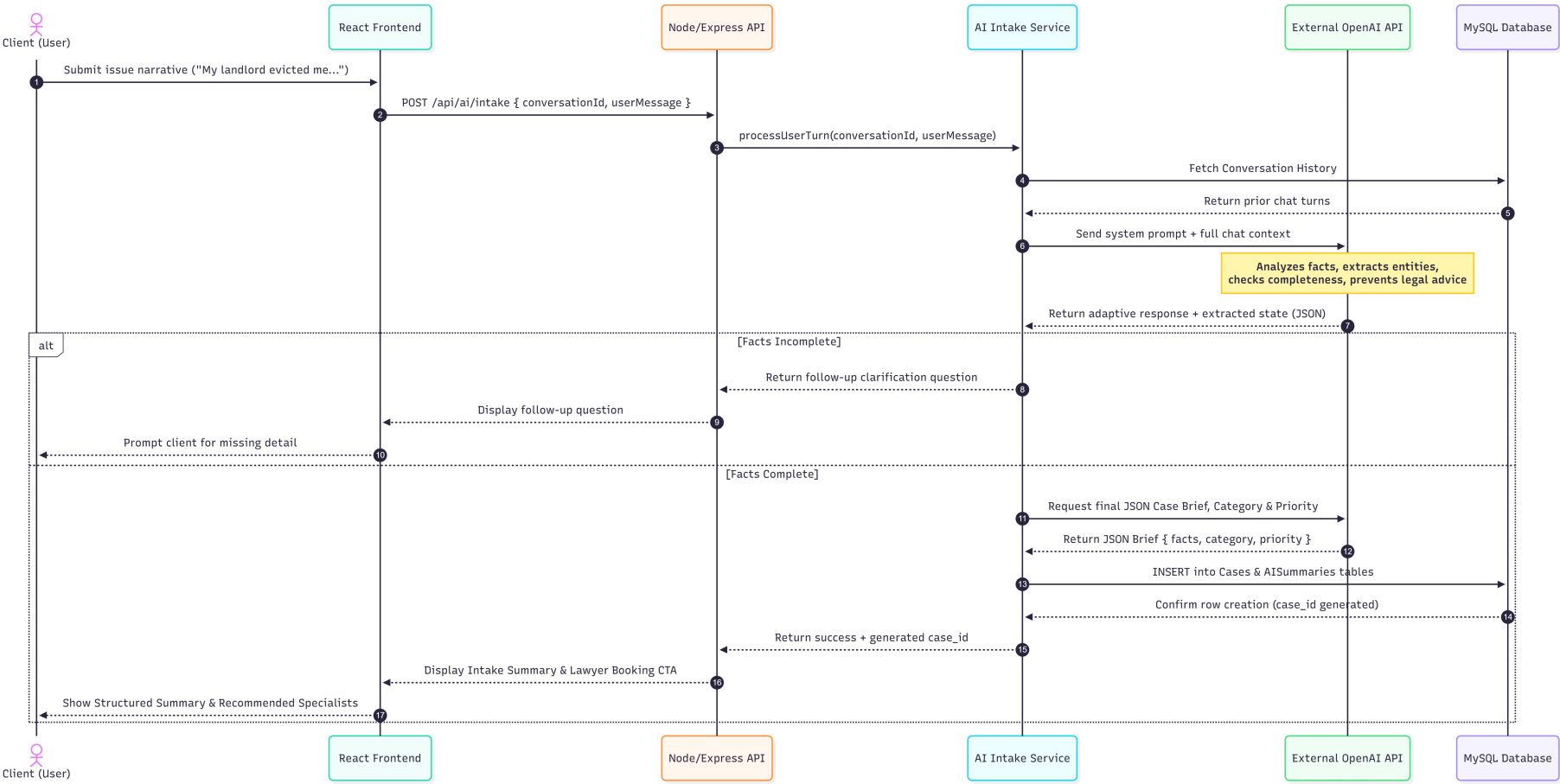
6. Sequence Diagram

Chosen Use Case: AI Legal Case Intake, Entity Extraction, Categorization, and Case Brief Generation (UC-2 through UC-5).

Participating Objects: Client (Actor), React Frontend (UI), Node/Express API, AI Intake Service, External OpenAI API, MySQL Database.

The diagram illustrates the full request/response chain that begins when a client submits a narrative description of their legal issue and ends with a structured case brief being persisted and shown back to the client along with recommended specialists.

Figure 3: Sequence Diagram — AI Legal Case Intake & Brief Generation



6.1 Sequence Diagram — Message Flow

- Client → UI: Submit issue narrative (“My landlord evicted me without notice…”).
- UI → API: POST /api/ai/intake { conversationId, userMessage }.
- API → AIService: processUserTurn(conversationId, userMessage).
- AIService → DB: Fetch ConversationHistory for conversationId.
- DB →→ AIService: Return prior chat turns list.
- AIService → OpenAI: Send system prompt + full chat context via REST API.
- OpenAI →→ AIService: Return adaptive response + extracted state (JSON).

Alt Condition — Fact Completeness Check

Branch A (Facts Incomplete):

- AIService →→ API: Return follow-up clarification question.
- API →→ UI: Display follow-up question widget.
- UI →→ Client: Prompt client for missing detail.

Branch B (Facts Complete):

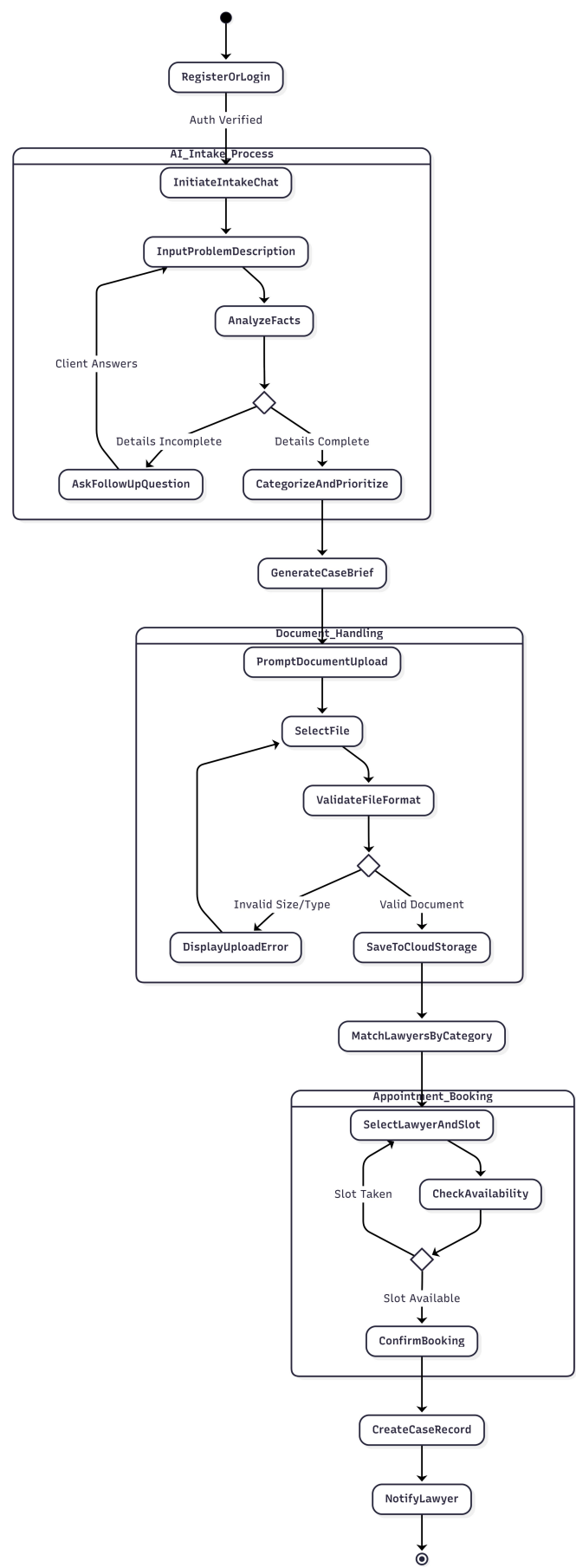
- AIService → OpenAI: Request final JSON Case Brief, Category, and Priority.
- OpenAI →→ AIService: Return JSON Brief { keyFacts, category, priority, entities }.
- AIService → DB: INSERT into Cases and INSERT into AISummaries.
- DB →→ AIService: Confirm row creation (returns generated case_id).
- AIService →→ API: Return success payload with case_id.
- API →→ UI: Display Intake Summary & Lawyer Booking CTA.
- UI →→ Client: Show Structured Summary & Recommended Specialists.

7. Activity Diagram

Chosen Process: End-to-End Client Intake, Document Processing, and Appointment Scheduling.

This diagram represents the complete client-facing workflow from authentication through AI-driven intake, evidence upload, and lawyer appointment booking, including three decision points that govern branching and retry loops within the process.

Figure 4: Activity Diagram — End-to-End Client Intake & Appointment Booking



7.1 Activity Diagram — Workflow Detail

Start: Client accesses the LexFlow web platform → registers/logs in via JWT authentication gateway.

Sub-Process: AI Intake

- Client enters natural language legal issue description.
- AI Intake Service processes dialogue and evaluates detail completeness.

Decision 1 — Are intake details complete?

- **No:** AI issues adaptive follow-up question → client answers → loop back to detail evaluation.
- **Yes:** System executes automated Case Categorization and Priority Assessment → generates structured AISummary case brief.

Sub-Process: Document Handling

- System prompts client to upload supporting legal files.
- File size and MIME type validation executed.

Decision 2 — Is the file format/size valid?

- **No:** Display error message → loop back to document upload step.
- **Yes:** Upload file to cloud storage and persist record in Documents table.
- System queries CaseCategories and displays matched specialist lawyers.

Sub-Process: Appointment Booking

- Client selects a lawyer and preferred calendar slot.
- System checks lawyer calendar availability.

Decision 3 — Is the selected slot available?

- **No:** Display slot conflict alert → loop back to slot selection.
- **Yes:** Confirm appointment reservation.

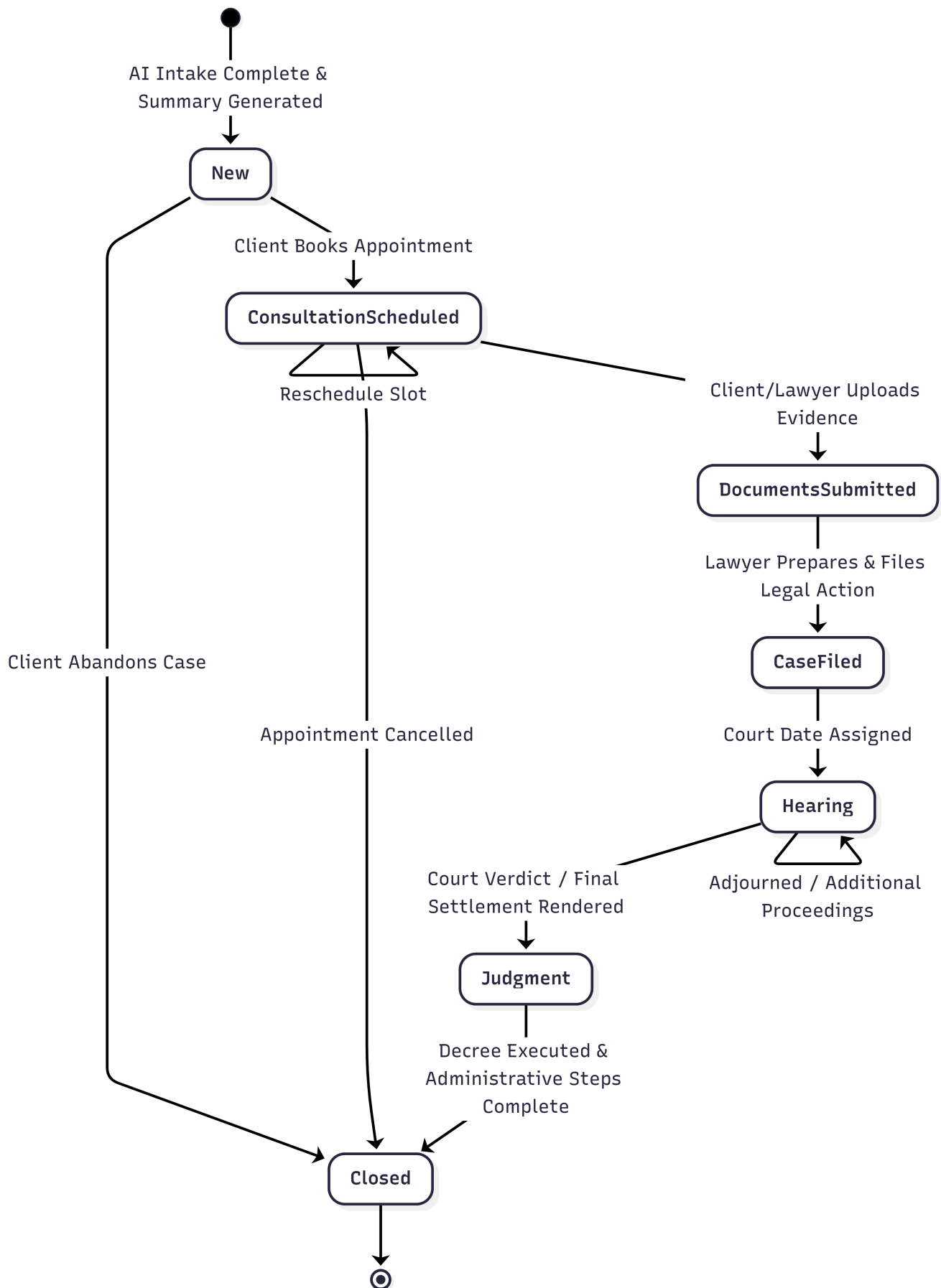
End: System writes final Case, AISummary, Document, and Appointment records to the database, dispatches email/dashboard notifications to the assigned lawyer, and the process completes with the case initialized in **Consultation Scheduled** status.

8. State Diagram

Chosen Object: Case Entity Lifecycle.

The diagram below traces a legal Case through seven lifecycle states, from initial AI-generated intake through consultation, filing, hearings, judgment, and closure, including alternate paths for early abandonment or cancellation.

Figure 5: State Diagram — Case Entity Lifecycle



8.1 State Diagram — States & Transitions

List of States

State	Description
New	Initial state created immediately after AI intake completion and summary generation.
Consultation Scheduled	Entered when client successfully books a consultation slot with a matched lawyer.
Documents Submitted	Entered when required evidentiary files are uploaded and verified.
Case Filed	Entered when lawyer prepares and files official court pleadings or legal notices.
Hearing	Entered when active trial proceedings or administrative court hearings begin.
Judgment	Entered when a final court verdict, judicial decree, or legal settlement is reached.
Closed	Final administrative state after execution of judgment or resolution of dispute.

State Transitions

- [*] → New | Trigger: AI intake conversation completes and AISummary row is created.
- New → Consultation Scheduled | Trigger: Client books appointment slot with matched lawyer.
- New → Closed | Trigger: Client abandons case prior to booking consultation.
- Consultation Scheduled → Documents Submitted | Trigger: Client or lawyer uploads required legal evidence files.
- Consultation Scheduled → Consultation Scheduled | Trigger: Appointment slot rescheduled by client or lawyer.
- Consultation Scheduled → Closed | Trigger: Appointment cancelled without rescheduling.
- Documents Submitted → Case Filed | Trigger: Lawyer submits official legal filings to court/opposing party.
- Case Filed → Hearing | Trigger: Court issues formal hearing or trial schedule date.
- Hearing → Hearing | Trigger: Hearing adjourned or additional trial session scheduled.

- Hearing → Judgment | Trigger: Court renders verdict or out-of-court settlement finalized.
- Judgment → Closed | Trigger: Judicial decree executed and administrative file finalized.
- Closed → [*] | Trigger: Case record archived.

9. Consistency Check

This documentation maintains full naming consistency across all system design specifications and UML diagrams:

- **Actor Names:** Client, Lawyer, Admin, External System: OpenAI API match identically across the Use Case descriptions, Sequence diagram, and Activity workflow.
- **Use Case Identifiers:** All 12 use cases (UC-1 through UC-12) match across the Use Case diagram, descriptions, and dependent workflow diagrams.
- **Class Entities:** The 11 classes (User, Client, Lawyer, Admin, Case, CaseCategory, AISummary, Document, Appointment, ConsultationNote, AuditLog) map directly to their corresponding database tables.
- **Lifecycle States:** The 7 case lifecycle states (New, Consultation Scheduled, Documents Submitted, Case Filed, Hearing, Judgment, Closed) are consistent between the State diagram and the underlying status enumeration.